

Intro

This is the main section of my submission, analyzing my results and reporting my findings. I will first detail my method, then go over some general observations I've made about how the AI agent performed, then analyze in detail some of the properties and test cases that stuck out to me.

Implementation

I wrote the structure identifier program by hand and ran it for each of the programs in the dataset. I prompted Cline via a shell script to generate a property JSON file based on each of these structure files. Finally, I prompted Cline in a similar way to generate a test program using Hypothesis for each of these property files. The full process took about 8 hours to run, with the majority of the time spent waiting for Cline to respond after prompting it.

General Observations

Trivial Properties

Many of the properties that the agent generated were very surface level. These properties just described the statements of the program, not revealing anything insightful to make useful tests out of. Indeed, most of the time these properties weren't even generated into tests by the agent, showing how trivial they were.

Interestingly, a majority of these trivial properties were branch-level. The function-level properties found by the agent are in general much more meaningful, revealing actual actionable properties that can be (and were) made into tests. This branch-level problem could be an error on my part, as I may not have provided an accurate definition of what I was expecting to be used as branch properties in my prompts. This doesn't mean that the function-level properties were free from triviality, though. A notable example is in `limit_ping_properties.json`, where one of the function level properties (`boolean_result`) simply declares that the output boolean will either have the value of true or false (see below).

```
{
  "scope": "function",
  "function": "limit_ping",
  "property": "boolean_result",
  "precondition": "window > 0 and limit >= 0",
  "formal": "limit_ping(timestamps, now, window=window, limit=limit)[0] in {True, False}"
},
```

Errors and Syntactic Issues

There was a significant amount of errors I had to fix to get all the tests to run properly. About 10 of the test programs had incorrect import statements for importing the program under test, and the other ~240 simply copied the program under test into the test program or used some path appending hacks to access the original file. I fixed the import errors with some

namespace changes on my machine, but I left the tests that copied the PUT as they were, after verifying that they didn't fix the bugs. Some other tests had errors related to using the `@example` annotation in Hypothesis, which I detailed in the part 2 report. After fixing these, all of the errors that entirely prevented tests from being run were fixed.

10 tests had warnings, but they still ran. All of these warnings were caused by the test using the `example()` function. Below is the warning message that Hypothesis output for these tests:

```
hypothesis.errors.HypothesisException: Using example() inside a test function is a bad idea. Instead consider using hypothesis.strategies.data() to draw more examples during testing.
```

After taking a look at the tests that caused this error, it's clear that even calling this function to generate a test case is unnecessary. Below is a characteristic example of the agent's use of this function:

```
@given(
    ratios=st.lists(st.floats(min_value=-1000, max_value=1000,
allow_nan=False, allow_infinity=False), min_size=1),
    fee=st.floats(min_value=0, max_value=1000, allow_nan=False,
allow_infinity=False)
)
def test_negative_amount_error(self, ratios, fee):
    """Test that negative amount raises ValueError('negative amount')."""
    negative_amount = st.floats(min_value=-1000, max_value=-0.001,
allow_nan=False, allow_infinity=False).example()

    with pytest.raises(ValueError, match="negative amount"):
        award_payout(negative_amount, ratios, fee=fee)
```

The test case could instead simply place the generator in the `@given` portion of the test case to avoid this warning and make the generation make more sense. This is one of many examples of the LLM making trivial, almost nonsensical mistakes in test and property generation.

As I mentioned in the part 2 PDF, there were 21 test suites (entire test programs) where a large majority (all or all except one) of the individual tests failed. Below is a table with all of the errors that caused these failures.

Test Name	Error	Explanation
batchnorm_running_mean_test	TypeError	positional arguments
budget_mix_test	TypeError	positional arguments
job_queue_priority_test	NameError	function not defined
log_sampling_decider_test	TypeError	positional arguments
packets_window_avg_test	TypeError	positional arguments
payment_split_rebate_test	TypeError	positional arguments

Test Name	Error	Explanation
quota_for_fuel_test	TypeError	positional arguments
realign_power_test	TypeError	module not callable
rent_splitter_test	TypeError	module not callable
reservation_overlap_check_test	TypeError	module not callable
revenue_share_test	TypeError	module not callable
schedule_call_test	NameError	function not defined
schedule_shift_allocator_test	NotImplementedError	inserted by the agent
schedule_window_test	AssertionError/ValueError	look at this more in detail
scores_apportion_test	TypeError	positional arguments
shares_apportion_test	TypeError	positional arguments
shift_demand_test	TypeError	module not callable
shift_risk_test	TypeError	positional arguments
shift_weight_test	NotImplementedError	inserted by the agent
storage_mix_test	TypeError	module not callable
votes_apportion_test	TypeError	positional arguments

A whopping 20 of these were trivial syntax errors, related once again to importing the program under test. All of the `TypeError`s are from either from incorrectly importing the *module* only instead of the *function* from the module, or from not specifying position arguments (e.g. calling the function like so: `shift_risk(current, target, damping)` instead of `shift_risk(current, target, damping=damping)`). The `NameError`s are a similar issue, where the test doesn't even attempt to import nor copy the function. The `NotImplementedError`s were purposely inserted by the AI agent. It defines a mock function that just raises this error for the test cases to call. I fix all of these in the next section to evaluate the quality of the tests.

The strangest error is in the `schedule_window_test`, where a majority of the `ValueError`s come from a simple mistake in how the function is called. It puts the input arguments the wrong way around, calling it with `interval`, existing when the right order is existing, `interval`.

I decided to fix most of these errors to get a more complete understanding of the quality of the agent's tests.

FailedHealthChecks

There are 20 test suites where over half of the failing tests are caused by `FailedHealthCheck` exceptions due to the test cases filtering out too many values. There's a lot more `FailedHealthCheck` exceptions scattered throughout the rest of the test suites, for a total of *259 out of 915 failing cases being caused by this same error*. This is a critical issue in the tests that my agent generated, as it's preventing all of these cases from actually testing the program. There were 8 test suites that had over 80 percent of their failing cases raise this error.

The `FailedHealthCheck` error is caused by the tests using the `assume()` function to filter out too many inputs that Hypothesis generates. All of the test suites that trigger these errors

have `assume()` calls that could instead be placed in the generator above each test (the `@given` part), without adding much complexity or changing what the test is testing for. It's a very simple mistake to fix, so it's surprising that the agent would make such a mistake, and that it was made so commonly that it caused almost a quarter of all of the test suites to fail. Due to its scale, this is an error I do not fix for the sake of time.

Logical Mistakes

There are a few instances where the tests the agent generated make logic errors that cause tests to either always pass or always fail, independent of the input. This obviously makes the tests effectively meaningless since they don't test the program properly.

For example, the test `test_uniform_bucket_distribution` in `log_sampling_decider_test.py` does not even call the program under test. Instead it checks if a randomly generated value creates a uniform distribution in the range `[0, 1)`. This *is* related to the original program, but will never fail. It's more of an environment test which verifies that the `hash()` function is working (which is quite unnecessary in this case).

Another example is `test_window_slicing` and `test_total_summation` in `packets_window_avg_test.py`. Both of these cases call the program under test, but don't use the value it returns. Instead they both do something like this:

```
recent = values[-window:]  
total = sum(recent)  
assert total == sum(recent),
```

in which the assertion will never fail. Meaning neither of these tests never produce a failing case, creating a false negative.

My last example shows that the agent can interpret a property wrong, making a mistake in the test. The tests `test_valid_ratios_property` and `test_non_negative_total_property` in `payment_split_rebate_test.py` have a similar problem as the previous example, where they call the program under test, but don't use its return value. However, this time, the test is intended to verify that the program under test does not raise errors for the inputs it feeds the function. It doesn't try to catch these errors though; the `assert` statements check whether the arguments it passed to the function are valid, but these arguments are already guaranteed to be valid with the generator and `assume()` calls.

Specific Analyses

In this section of the report, I will review the AI Agent's semantic property and test generation, sampled from the corpus of programs that were given for this project. I don't think you want to sift through 250 reviews, so instead I will select a few program results to analyze (including both properties and tests) that produce the interesting results after running both the property and test generation.

Specific Analysis #1: normalize.py

Properties

```
"scope": "branch",
"function": "normalize",
"condition": "len(xs) == 0",
"property": "identity_on_empty",
"formal": "normalize(xs) == xs"

"scope": "branch",
"function": "normalize",
"condition": "len(xs) != 0",
"property": "unit_sum",
"formal": "sum(normalize(xs)) == 1"

"scope": "branch",
"function": "normalize",
"condition": "len(xs) != 0 and all(x >= 0 for
x in xs)",
"property": "non_negative_outputs",
"formal": "all(x >= 0 for x in normalize(xs))"

"scope": "function",
"function": "normalize",
"property": "unit_sum",
"precondition": "sum(xs) != 0",
"formal": "sum(normalize(xs)) == 1"

"scope": "function",
"function": "normalize",
"property": "non_negative_outputs",
"precondition": "all(x >= 0 for x in xs)",
"formal": "all(x >= 0 for x in normalize(xs))"

"scope": "function",
"function": "normalize",
"property": "scale_invariance",
"precondition": "c != 0",
"formal": "normalize([c * x for x in xs]) ==
normalize(xs)"

"scope": "function",
"function": "normalize",
"property": "identity_on_empty",
"precondition": "len(xs) == 0",
"formal": "normalize(xs) == xs"
```

These properties are perfectly fine for the simple normalize function, but they don't differentiate between branch level and function level properties enough. Many of the branch-level and function-level properties say the same thing. For example, branch property 2 and function property 1 both say that the output will sum to 1, and that the list must be nonempty for that to happen. Branch property 3 and function property 2 say that there will be non-negative outputs when the input is non-negative and nonempty.

The AI only notices that an input list of [0.0] will result in a ZeroDivisionError in one of the properties, the function property unit_sum. Many of the other properties omit this precondition which causes the corresponding test to fail. We can refine this by adding to unit_sum, non_negative_outputs, and scale_invariance the following precondition: all(x > 0 for x in normalize(xs)) (note the lack of '=').

This bug being caught like this with failing tests *could* be seen as a good thing, as in the tests are doing their job, but it's still an inconsistency in how the AI reasoned about the original program. If it added the condition to one property, it probably should have added it to all the other ones.

Tests – File Location: normalize/test_normalize_retry.py

Cline does not consider bounding the input floats, causing certain tests to experience division by zero errors, which don't reveal much about the original program other than it shouldn't be given very very large floats. Some of the tests that were generated *do* indeed reveal some problems with the original function, specifically test_non_negative_inputs, which reveals that there is a ZeroDivisionError when the input is one or more zeros. The revealed error albeit wasn't part of the semantic properties that the agent identified, so it could have done better.

Specific Analysis #2: replica_sync_offsets.py

Properties

This function is supposed to find values in the primary list that aren't in the replica list, and add them to the replica. Below are the properties:

```
"scope": "branch",
"function": "replica_sync_offsets",
"condition": "primary[i] < replica[j]",
"property": "missing_element_identified",
"formal": "missing.append(primary[i])",

"scope": "branch",
"function": "replica_sync_offsets",
"condition": "replica[j] < primary[i]",
"property": "replica_skip",
"formal": "j += 1",

"scope": "branch",
"function": "replica_sync_offsets",
"condition": "not (primary[i] < replica[j] and replica[j] <
primary[i])",
"property": "common_element_found",
"formal": "i += 1 and j += 1",

"scope": "branch",
"function": "replica_sync_offsets",
"condition": "missing and replica and missing[-1] ==
replica[-1]",
"property": "duplicate_removal",
"formal": "missing.pop()",

"scope": "function",
"function": "replica_sync_offsets",
"property": "monotonic_sequences",
"precondition": "primary and replica are sorted in ascending
order",
"formal": "for all k, primary[k] <= primary[k+1] and replica[k]
<= replica[k+1]"
```

Some of the properties the agent identified were technically correct, but don't give much insight into the program's actual usable properties. They are surface level, explaining the structure of the program without making any other insight. For example, one of the generated branch-level properties is called "replica_skip", and it essentially just says that $j += 1$ when $replica[j] < primary[i]$. This *is* correct, but is simply the line of code in the branch and doesn't reveal any meaningful or testable property about the program. The same thing is true for the properties `missing_element_identified` and `common_element_found`.

The "monotonic_sequences" property formalizes the requirement of the two lists to be sorted, but this is not so much a property about the program as it is an *assumed characteristic* about the input that the program *uses*. So, this property is inconsequential.

Tests - File Location: tests/replica_sync_offsets_test.py

The edge case test (below) doesn't properly test if the generated list is correct, as it assumes duplicates are not counted, but they are. The example of using duplicate elements in both the primary and replica lists causes a test to fail. This is a mistake in the implementation of the test that fails to notice a property of the program.

```
def test_edge_cases(self, primary, replica):
    primary_sorted = sorted(primary)
    replica_sorted = sorted(replica)

    result = replica_sync_offsets(primary_sorted, replica_sorted)

    # Verify basic properties
    assert isinstance(result, list)
    assert len(result) <= len(primary_sorted)

    # Verify that result contains only elements from primary not in replica
    for element in result:
        assert element in primary_sorted
        assert element not in replica_sorted
```

To refine this test, we can change the last for loop to this:

```
for element in result:
    if element in replica_sorted:
        # should be a duplicate in the primary
        assert primary_sorted.count(element) > replica_sorted.count(element)
    else:
        assert element in primary_sorted
```

Most of the other tests in the file make the same mistake, so I fixed them too.

Specific Analysis #3: award_payout.py

Properties

This function divides an award pool (list) by a list of ratios, optionally deducting a fee from each participant. The properties that were generated are as follows:

```
"scope": "branch",
"function": "award_payout",
"condition": "not ratios",
"property": "empty_ratios_error",
"formal": "award_payout(amount, []) raises
ValueError('ratios required')"
```

```
"scope": "branch",
"function": "award_payout",
"condition": "amount < 0",
"property": "negative_amount_error",
"formal": "award_payout(negative_amount,
ratios) raises ValueError('negative amount')"
```

```
"scope": "branch",
"function": "award_payout",
"condition": "total_ratio <= 0",
"property": "invalid_ratios_error",
"formal": "award_payout(amount, ratios) raises
ValueError('invalid ratios') when sum(ratios) <=
0"
```

```
"scope": "function",
"function": "award_payout",
"property": "fee_deduction",
"precondition": "ratios is not empty and
amount >= 0 and sum(ratios) > 0",
"formal": "all(s == (r / sum(ratios)) * amount
- fee for s, r in zip(award_payout(amount,
ratios), ratios))"
```

```
"scope": "function",
"function": "award_payout",
"property": "proportional_distribution",
"precondition": "ratios is not empty and
amount >= 0 and sum(ratios) > 0 and fee == 0",
"formal": "sum(award_payout(amount, ratios))
== amount"
```

```
"scope": "function",
"function": "award_payout",
"property": "non_negative_shares",
"precondition": "ratios is not empty and
amount >= 0 and sum(ratios) > 0 and fee <= min((r
/ sum(ratios)) * amount for r in ratios)",
"formal": "all(s >= 0 for s in
award_payout(amount, ratios))"
```

```
"scope": "function",
"function": "award_payout",
"property": "length_preservation",
"precondition": "ratios is not empty and
amount >= 0 and sum(ratios) > 0",
"formal": "len(award_payout(amount, ratios))
== len(ratios)"
```

Most of the properties that were generated for this function are quite trivial. The first three (`empty_ratios_error`, `negative_amount_error`, and `invalid_ratios_error`) simply reiterate the conditions that raise errors, which can be easily observed from looking at the original function. Of the other four function properties, three of them are meaningful and one isn't. `proportional_distribution`, `length_preservation`, and `non_negative_shares` are all non-trivial and reveal properties about the program. `fee_deduction` is just the logic of the program formatted in a less readable way. It *could* be useful as an oracle, and it does always

hold as long as the preconditions are met, but it also doesn't contain any insights about what the original function is doing.

Tests – File Location: tests/award_payout_test.py

In terms of tests, each of the properties are turned into a single test, which is generally good practice. It doesn't try to design any specific test for edge cases, which is also good because Hypothesis tends to find edge cases automatically. Two of the tests use the `.example()` method to generate a value for one of the arguments to the `award_payout` function. These usages of this method don't really make sense; the argument could simply be generated outside of the test function in the `@given` line. They cause Hypothesis to generate a warning and not execute the tests with the `example()` call present. The rest of the tests pass. I refined the test functions by removing these, which causes all of the tests to pass.

Specific Analysis #4: votes_apportion.py

This function applies a proportion based on weights to each element in the total list. If the final value (proportion * total) is less than a minimum, it raises it to this minimum. If the `floor_to_int` flag is set, every value in the final list is floored to an int value, *even if this is below the minimum*. The program has a comment indicating a bug, saying it “floors without distributing the remainder”, which is more of a specification error than a logic error (similar to `award_payout`). Below are a few of the properties that the agent generated:

```
"scope": "function",
"function": "votes_apportion",
"property": "non_empty_weights_precondition",
"precondition": "len(weights) > 0",
"formal": "len(weights) > 0"

"scope": "function",
"function": "votes_apportion",
"property": "non_negative_total_precondition",
"precondition": "total >= 0",
"formal": "total >= 0"

"scope": "function",
"function": "votes_apportion",
"property":
"non_negative_minimum_precondition",
"precondition": "minimum >= 0",
"formal": "minimum >= 0"

"scope": "function",
"function": "votes_apportion",
"property":
"non_zero_weight_sum_precondition",
"precondition": "sum(weights) > 0",
"formal": "sum(weights) > 0"
```

These properties are seemingly only there to define preconditions for other properties, without declaring anything by themselves. They do this even though there is a specific precondition field in the property format, where properties like this can be defined. All four of these properties were used to generate a test which is supposed to ensure the precondition doesn't cause a failure. An example of one these tests is below (they all share the same format):

Tests – File Location: tests/votes_apportion_test.py

```
@given(
    total=st.floats(min_value=0, max_value=1000, allow_nan=False, allow_infinity=False),
    weights=st.lists(st.floats(min_value=0, max_value=1000, allow_nan=False, allow_infinity=False),
min_size=1, max_size=100).filter(lambda w: sum(w) > 0),
    minimum=st.floats(min_value=0, max_value=1000, allow_nan=False, allow_infinity=False),
    floor_to_int=st.booleans())
```

```
)
def test_non_negative_total_precondition(self, total, weights, minimum, floor_to_int):
    """
    Test Property: non_negative_total_precondition
    Precondition: total >= 0
    Formal: total >= 0
    """
    assume(sum(weights) > 0)
    # This is a precondition test - the function should work when precondition is met
    result = votes_apportion(total, weights, minimum=minimum, floor_to_int=floor_to_int)
    assert total >= 0, "Precondition violated: total should be non-negative"
```

These tests don't check for failure properly. They want to ensure the function works, but don't try to catch any errors. The assert statement the function is called will always be true for all of these tests. They only cause a failure when an error is raised in the execution of the function, which we already test for in the branch property tests above! So, these tests are largely unnecessary.

```
@given(
    total=st.floats(min_value=0, max_value=1000, allow_nan=False, allow_infinity=False),
    weights=st.lists(st.floats(min_value=0, max_value=1000, allow_nan=False, allow_infinity=False),
min_size=1, max_size=100).filter(lambda w: sum(w) > 0),
    minimum=st.floats(min_value=0, max_value=1000, allow_nan=False, allow_infinity=False),
    floor_to_int=st.booleans()
)
def test_shares_minimum_bound_invariant(self, total, weights, minimum, floor_to_int):
    """
    Test Property: shares_minimum_bound_invariant
    Precondition: len(weights) > 0 and total >= 0 and minimum >= 0 and sum(weights) > 0
    Formal: all(s >= minimum for s in shares)
    """
    result = votes_apportion(total, weights, minimum=minimum, floor_to_int=floor_to_int)
    if floor_to_int:
        minimum = int(minimum) # refined branch and check
    assert all(s >= minimum for s in result), f"Minimum bound invariant failed: found value below
minimum {minimum} in {result}"
```

Moving on to the more meaningful tests, `test_shares_minimum_bound_invariant` (above) does not consider the `floor_to_int` flag when it does its assertion. It only checks to see if the float minimum exists in the list even though it would be floored if `floor_to_int` is True. This can be refined by adding another branch which checks if `floor_to_int` is set. Another test, `test_shares_sum_bound_invariant` is only valid when `floor_to_int` is False. So we can refine it by changing the precondition (below). Both of these are examples of the agent not including guards against conditional program behavior to ensure the tests pass, showing that AI generated tests can sometimes miss critical details about program behavior.

```
@given(
    total=st.floats(min_value=0, max_value=1000, allow_nan=False, allow_infinity=False),
    weights=st.lists(st.floats(min_value=0, max_value=1000, allow_nan=False,
allow_infinity=False), min_size=1, max_size=100).filter(lambda w: sum(w) > 0),
    minimum=st.floats(min_value=0, max_value=1000, allow_nan=False, allow_infinity=False),
    floor_to_int=st.booleans().filter(lambda ft: ft == False) # previously just st.booleans()
)
def test_shares_sum_bound_invariant(self, total, weights, minimum, floor_to_int):
    """
    Test Property: shares_sum_bound_invariant
```

```
Precondition: len(weights) > 0 and total >= 0 and minimum >= 0 and sum(weights) > 0 and
floor_to_int = False
Formal: sum(shares) >= total
"""
result = votes_apportion(total, weights, minimum=minimum, floor_to_int=floor_to_int)
assert sum(result) >= total, f"Sum bound invariant failed: sum({result}) < {total}"
```

Takeaways

I learned a lot about using AI for testing in this project. Firstly, *it doesn't always do what you want*. There are a lot of errors and mistakes that can surface from seemingly benign requests. There were very few test programs that the agent initially generated which correctly imported the program they were trying to test. Many of the test suites that didn't import programs correctly also called them with the wrong signature. Tests would run, but they would automatically fail due to a runtime error. This was a very common occurrence, with over 20 tests having all of their test cases fail because of this issue. It's not difficult to fix, but it is tedious; requiring a developer to find the failing cases and add the import statements, replace the function signatures, or move statements around.

In my opinion, the most egregious issue in the tests that the AI generated were how many caused a FailedHealthCheck error due to filtering out too many inputs in the test function. About a *quarter* of the failing individual test cases were caused by this issue. It *can* be circumvented by changing the Hypothesis settings, but it'll cause all the tests to run slower as the engine needs to generate even more inputs to the functions. The test cases already take about 7 minutes to finish running, and that's for a relatively small source codebase (250 files, about 20 lines each, only around 5000 lines of code).

I also learned how inconsistent the test and property generation is when done by an agent. The same prompt on a different program would lead to slightly different results. Different kinds of properties would be generated, test suites would be organized differently, certain test suites would have unique issues, et cetera. This doesn't make troubleshooting impossible, just harder than it would be if all the tests had a strict format. Better prompting when instructing the AI to generate tests may allow the agent to do something like this, though.

There were a significant number of suites that included edge case tests, where the case would define examples that tested specific parts of the source program. This was odd to me, as Hypothesis is designed to automatically find edge cases through generating random values with the generators that are given to it. This, combined with the agent's misuse of other Hypothesis parts like `.example()` and the many FailedHealthCheck errors, indicates to me that AI agents cannot reliably generate well-formed tests that follow any given tool's semantics. They aren't completely reliable for in depth code generation at this point in time.

Overall, this project showed me that AI agents aren't *quite* ready to be used to automatically generate tests for small scale programs, let alone projects with bigger codebases. Sure, it was able to save the developer time writing all the properties and tests, but I think fixing all the errors (subtle and obvious) in these tests would certainly take a similar amount of time, without even considering the quality of the tests. Perhaps a different prompt, or a different model, would provide better results, but at this point in time I'm confident that LLMs should be

Garett Hammerle
CS 206 - Winter 2026

used mainly for making small-scale queries and for learning, not for larger in-depth tasks like entire test suite generation.